PROJETO VFS – *Orion68DOS*

Tudo é um Arquivo - Implementação Completa
Data: Agosto 2026 - Versão: 1.0

1. INTRODUÇÃO

1.1 Objetivo

Implementar o paradigma *"Tudo é um arquivo"*no sistema operacional Orion68DOS (m68k), permitindo que dispositivos, informações do sistema e arquivos reais sejam acessados através das mesmas funções padronizadas.

1.2 Conceito

No Linux/Unix, tudo pode ser tratado como um arquivo:

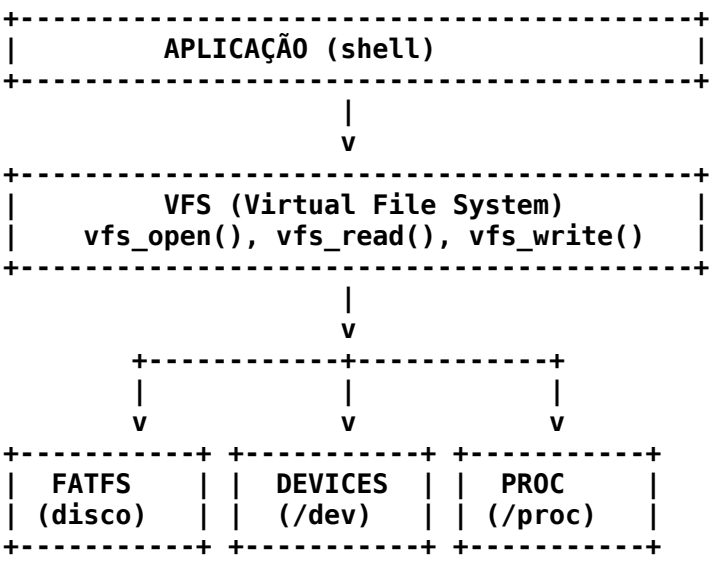
Tipo	Exemplo	Descrição
Arquivos de disco	/boot.bat	Dados armazenados em FATFS
Dispositivos de hardware	/dev/tty	Terminal (vídeo/teclado)
Dispositivos virtuais	/dev/null	Descarta dados
Informações do sistema	/proc/meminfo	Memória disponível

1.3 Benefícios

- Padronização: Uma única API para tudo
- Simplicidade: Código mais limpo e reutilizável
- Flexibilidade: Fácil adicionar novos dispositivos
- Redirecionamento: Suporte a > e < no shell
- Pipes: Base para comunicação entre processos (futuro)

2. ARQUITETURA DO SISTEMA

2.1 Visão Geral



2.2 Camadas do Sistema

Camada 1: Aplicação (Shell)

- Interface com o usuário

- Comandos como cat, echo, ls
- Não sabe se está lendo de arquivo, TTY ou /proc

Camada 2: VFS (Virtual File System)
- Gerencia descritores de arquivos (FD)
- Roteia chamadas para o driver correto
- Mantém tabela de arquivos abertos

Camada 3: Drivers
- FATFS: Arquivos em disco (usa libfileio)
- TTY: Terminal (getchar/putchar)
- NULL: Dispositivo nulo
- ZERO: Gerador de zeros
- PROC: Informações do sistema

Camada 4: Hardware/Sistema
- SD Card (via FATFS)
- UART/Teclado (via getchar)
- Vídeo (via putchar)
- Memória/CPU (para /proc)

3. ESTRUTURAS DE DADOS

3.1 Estrutura File

```
typedef struct File
    char *name;
    int type;
    size_t position;
    int ref_count;
    void *private_data;
    int (*open)(struct File *file, const char *path, int flags);
    int (*read)(struct File *file, void *buffer, size_t size);
    int (*write)(struct File *file, const void *buffer, size_t size);
    int (*close)(struct File *file);
    int (*ioctl)(struct File *file, int cmd, void *arg);
    size_t (*lseek)(struct File *file, size_t offset, int whence);
 File;
```

3.2 Estrutura DeviceDriver

```
typedef struct
    const char *path;
    int (*open)(File *, const char *, int);
    int (*read)(File *, void *, size_t);
    int (*write)(File *, const void *, size_t);
    int (*close)(File *);
    int (*ioctl)(File *, int, void *);
    size_t (*lseek)(File *, size_t, int);
 DeviceDriver;
```

3.3 Tabela de Descritores

Descritor	Uso	Dispositivo
0	stdin (entrada padrão)	/dev/tty
1	stdout (saída padrão)	/dev/tty
2	stderr (saída de erro)	/dev/tty

4. DRIVERS IMPLEMENTADOS

4.1 Driver FATFS (vfs_fat.c)

Função: Acessar arquivos no sistema FATFS
Dependências:libfileio.a (suas syscalls)
Operações:

Função	Syscall usada	Descrição
fat_open()	fopen()	Abre arquivo no disco
fat_read()	fread()	Lª do arquivo
fat_write()	fwrite()	Escreve no arquivo
fat_close()	fclose()	Fecha o arquivo
fat_lseek()	flseek()	Move posição

4.2 Driver TTY (vfs_tty.c)

Função: Terminal de texto (vídeo/teclado)
Dependências:getchar(), putchar() do sistema
Operações:

Função	Descrição
tty_open()	Sempre sucesso
tty_read()	Lª do teclado (getchar) com echo
tty_write()	Escreve no vídeo (putchar)
tty_close()	Sempre sucesso
tty_ioctl()	Comandos especiais (limpar tela)
tty_lseek()	Não suportado (retorna -1)

4.3 Driver NULL (vfs_null.c)

Função: Dispositivo nulo (descarta dados)

4.4 Driver ZERO (vfs_zero.c)

Função: Gerador de zeros

4.5 Driver PROC (vfs_proc.c)

Função: Informações do sistema
Arquivos suportados:

Arquivo	Conteúdo
/proc/meminfo	Informações de memória
/proc/cpuinfo	Informações da CPU
/proc/uptime	Tempo desde o boot

5. API DO VFS

5.1 vfs_open()

```
int vfs_open(const char *path, int flags);
```

Parâmetros:

Parâmetro	Tipo	Descrição
path	const char*	Caminho do arquivo/dispositivo
flags	int	Flags de abertura (O_RDONLY, O_WRONLY, etc.)

Retorno:

Valor Significado
>= 0 Descritor de arquivo (FD)
-1 Erro (arquivo não encontrado)
Exemplo:
```
int fd = vfs_open("/boot.bat", O_RDONLY);
if (fd >= 0)
    // Arquivo aberto com sucesso
```

5.2 vfs_read()
```
int vfs_read(int fd, void *buffer, size_t size);
```

5.3 vfs_write()
```
int vfs_write(int fd, const void *buffer, size_t size);
```

5.4 vfs_close()
```
int vfs_close(int fd);
```

5.5 vfs_ioctl()
```
int vfs_ioctl(int fd, int cmd, void *arg);
```

5.6 vfs_lseek()
```
size_t vfs_lseek(int fd, size_t offset, int whence);
```

6. SHELL E COMANDOS
6.1 Comandos Implementados
Comando	Função	Descrição
help	cmd_help()	Lista comandos disponíveis
echo	cmd_echo()	Escreve texto no stdout
cat	cmd_cat()	Exibe conteúdo de arquivo
ls	cmd_ls()	Lista arquivos do diretório
cd	cmd_cd()	Muda diretório atual
mkdir	cmd_mkdir()	Cria um diretório
reboot	cmd_reboot()	Reinicia o sistema
shutdown	cmd_shutdown()	Desliga o sistema
meminfo	cmd_meminfo()	Mostra informações de memória
clear	cmd_clear()	Limpa a tela

6.2 Exemplos de Uso
Exemplo 1: Lendo um arquivo
```
> cat /boot.bat
# Script de inicialização do Orion68DOS
echo "Bem-vindo!"
meminfo
```

Exemplo 2: Escrevendo no terminal
```
> echo "Hello World"
Hello World
```

Exemplo 3: Lendo informações do sistema

```
> cat /proc/meminfo
MemTotal: 1024 KB
MemFree:  512 KB
MemUsed:  512 KB
MemAvail: 256 KB
```

7. CÓDIGO FONTE COMPLETO

7.1 vfs.h

```
#ifndef VFS_H
#define VFS_H

#include <stdint.h>
#include <stddef.h>
#include <fatfs/ff.h>

#define FILE_TYPE_REGULAR   0
#define FILE_TYPE_DEVICE    1
#define FILE_TYPE_PROC      2

#define O_RDONLY    0x01
#define O_WRONLY    0x02
#define O_RDWR      0x03
#define O_CREAT     0x04
#define O_TRUNC     0x08
#define O_APPEND    0x10

#define TIOC_CLEAR  0x01
#define TIOC_GETXY  0x02
#define TIOC_SETXY  0x03

typedef struct File
    char *name;
    int type;
    size_t position;
    int ref_count;
    void *private_data;
    int (*open)(struct File *file, const char *path, int flags);
    int (*read)(struct File *file, void *buffer, size_t size);
    int (*write)(struct File *file, const void *buffer, size_t size);
    int (*close)(struct File *file);
    int (*ioctl)(struct File *file, int cmd, void *arg);
    size_t (*lseek)(struct File *file, size_t offset, int whence);
 File;

typedef struct
    const char *path;
    int (*open)(File *file, const char *path, int flags);
    int (*read)(File *file, void *buffer, size_t size);
    int (*write)(File *file, const void *buffer, size_t size);
    int (*close)(File *file);
    int (*ioctl)(File *file, int cmd, void *arg);
    size_t (*lseek)(File *file, size_t offset, int whence);
 DeviceDriver;
```

```
int vfs_open(const char *path, int flags);
int vfs_read(int fd, void *buffer, size_t size);
int vfs_write(int fd, const void *buffer, size_t size);
int vfs_close(int fd);
int vfs_ioctl(int fd, int cmd, void *arg);
size_t vfs_lseek(int fd, size_t offset, int whence);

int allocate_fd(File *file);
File *get_file_from_fd(int fd);
void free_fd(int fd);
void vfs_init(void);

#endif
```

7.2 vfs_fat.c

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "vfs.h"
#include "fileio.h"

typedef struct
    FIL *fat_file;
    BYTE mode;
    FSIZE_t size;
 FatPrivate;

int fat_open(File *file, const char *path, int flags)
    FIL *fat_file = (FIL*)malloc(sizeof(FIL));
    if (!fat_file) return -1;
    BYTE fat_mode = 0;
    if (flags & O_RDONLY) fat_mode |= FA_READ;
    if (flags & O_WRONLY) fat_mode |= FA_WRITE;
    if (flags & O_RDWR) fat_mode |= FA_READ | FA_WRITE;
    if (flags & O_CREAT) fat_mode |= FA_CREATE_ALWAYS;
    if (flags & O_APPEND) fat_mode |= FA_OPEN_APPEND;
    FRESULT result = fopen(fat_file, path, fat_mode);
    if (result != FR_OK)  free(fat_file); return -1;
    FatPrivate *priv = (FatPrivate*)malloc(sizeof(FatPrivate));
    if (!priv)  fclose(fat_file); free(fat_file); return -1;
    priv->fat_file = fat_file;
    priv->mode = fat_mode;
    priv->size = fsize(fat_file);
    file->private_data = priv;
    file->position = 0;
    file->read = fat_read;
    file->write = fat_write;
    file->close = fat_close;
    file->lseek = fat_lseek;
    return 0;


int fat_read(File *file, void *buffer, size_t size)
    FatPrivate *priv = (FatPrivate*)file->private_data;
```

```
    if (!priv) return -1;
    UINT bytes_read;
    FRESULT result = fread(priv->fat_file, buffer, (UINT)size, &bytes_read);
    if (result != FR_OK) return -1;
    file->position += bytes_read;
    return (int)bytes_read;


int fat_write(File *file, const void *buffer, size_t size)
    FatPrivate *priv = (FatPrivate*)file->private_data;
    if (!priv) return -1;
    UINT bytes_written;
    FRESULT result = fwrite(priv->fat_file, buffer, (UINT)size,
&bytes_written);
    if (result != FR_OK) return -1;
    file->position += bytes_written;
    if (file->position > priv->size) priv->size = file->position;
    return (int)bytes_written;


int fat_close(File *file)
    FatPrivate *priv = (FatPrivate*)file->private_data;
    if (!priv) return -1;
    FRESULT result = fclose(priv->fat_file);
    free(priv->fat_file); free(priv);
    file->private_data = NULL;
    return (result == FR_OK) ? 0 : -1;


size_t fat_lseek(File *file, size_t offset, int whence)
    FatPrivate *priv = (FatPrivate*)file->private_data;
    if (!priv) return (size_t)-1;
    FSIZE_t new_pos;
    FSIZE_t current = ftell(priv->fat_file);
    switch (whence)
        case 0: new_pos = (FSIZE_t)offset; break;
        case 1: new_pos = current + (FSIZE_t)offset; break;
        case 2: new_pos = priv->size + (FSIZE_t)offset; break;
        default: return (size_t)-1;

    FRESULT result = flseek(priv->fat_file, new_pos);
    if (result != FR_OK) return (size_t)-1;
    file->position = (size_t)new_pos;
    return (size_t)new_pos;
```

7.3 vfs_tty.c

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "vfs.h"

extern char getchar(void);
extern void putchar(char c);
```

```
int tty_open(File *file, const char *path, int flags)
    file->private_data = NULL;
    file->position = 0;
    file->read = tty_read;
    file->write = tty_write;
    file->close = tty_close;
    file->ioctl = tty_ioctl;
    file->lseek = tty_lseek;
    return 0;


int tty_read(File *file, void *buffer, size_t size)
    char *buf = (char*)buffer;
    size_t i = 0;
    while (i < size)
        char c = getchar();
        buf[i++] = c;
        putchar(c);
        if (c == '' || c == '')  putchar(''); break;

    return (int)i;


int tty_write(File *file, const void *buffer, size_t size)
    const char *data = (const char*)buffer;
    for (size_t i = 0; i < size; i++)
        if (data[i] == '') putchar('');
        putchar(data[i]);

    return (int)size;


int tty_close(File *file)  return 0;

int tty_ioctl(File *file, int cmd, void *arg)
    switch (cmd)
        case TIOC_CLEAR: printf("\033[2J\033[H"); return 0;
        default: return -1;



size_t tty_lseek(File *file, size_t offset, int whence)
    return (size_t)-1;
```

7.4 vfs_null.c

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "vfs.h"

int null_open(File *file, const char *path, int flags)
    file->private_data = NULL;
    file->position = 0;
    file->read = null_read;
```

```
    file->write = null_write;
    file->close = null_close;
    file->ioctl = NULL;
    file->lseek = null_lseek;
    return 0;


int null_read(File *file, void *buffer, size_t size)  return 0;
int null_write(File *file, const void *buffer, size_t size)  return
(int)size;
int null_close(File *file)  return 0;
size_t null_lseek(File *file, size_t offset, int whence)  return offset;
```

7.5 vfs_zero.c

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "vfs.h"

int zero_open(File *file, const char *path, int flags)
    file->private_data = NULL;
    file->position = 0;
    file->read = zero_read;
    file->write = zero_write;
    file->close = zero_close;
    file->ioctl = NULL;
    file->lseek = zero_lseek;
    return 0;


int zero_read(File *file, void *buffer, size_t size)
    memset(buffer, 0, size);
    file->position += size;
    return (int)size;


int zero_write(File *file, const void *buffer, size_t size)  return
(int)size;
int zero_close(File *file)  return 0;

size_t zero_lseek(File *file, size_t offset, int whence)
    switch (whence)
        case 0: file->position = offset; break;
        case 1: file->position += offset; break;
        case 2: file->position = offset; break;
        default: return (size_t)-1;

    return file->position;
```

7.6 vfs_proc.c

```
#include <stdio.h>
#include <string.h>
```

```
#include <stdlib.h>
#include "vfs.h"

typedef struct
    char *data;
    size_t size;
    size_t position;
 ProcData;

int get_total_memory(void)  return 1024;
int get_free_memory(void)  return 512;
const char *get_cpu_name(void)  return "Motorola 68000";
int get_cpu_clock(void)  return 8;

int proc_read(File *file, void *buffer, size_t size)
    ProcData *data = (ProcData*)file->private_data;
    if (!data) return -1;
    size_t available = data->size - data->position;
    size_t to_read = (size < available) ? size : available;
    if (to_read == 0) return 0;
    memcpy(buffer, data->data + data->position, to_read);
    data->position += to_read;
    file->position = data->position;
    return (int)to_read;


int proc_close(File *file)
    ProcData *data = (ProcData*)file->private_data;
    if (data)  free(data->data); free(data); file->private_data = NULL;
    return 0;


size_t proc_lseek(File *file, size_t offset, int whence)
    ProcData *data = (ProcData*)file->private_data;
    if (!data) return (size_t)-1;
    size_t new_pos;
    switch (whence)
        case 0: new_pos = offset; break;
        case 1: new_pos = data->position + offset; break;
        case 2: new_pos = data->size + offset; break;
        default: return (size_t)-1;

    if (new_pos > data->size) new_pos = data->size;
    data->position = new_pos;
    file->position = new_pos;
    return new_pos;


int proc_meminfo_open(File *file, const char *path, int flags)
    ProcData *data = (ProcData*)malloc(sizeof(ProcData));
    if (!data) return -1;
    char buffer[256];
    sprintf(buffer,
        "MemTotal: %d KB\nMemFree: %d KB\nMemUsed: %d KB\nMemAvail: %d KB\n",
        get_total_memory(), get_free_memory(),
        get_total_memory() - get_free_memory(), get_free_memory() / 2);
    data->data = strdup(buffer);
```

```
    data->size = strlen(buffer);
    data->position = 0;
    file->private_data = data;
    file->position = 0;
    file->read = proc_read;
    file->write = NULL;
    file->close = proc_close;
    file->ioctl = NULL;
    file->lseek = proc_lseek;
    return 0;


int proc_cpuinfo_open(File *file, const char *path, int flags)
    ProcData *data = (ProcData*)malloc(sizeof(ProcData));
    if (!data) return -1;
    char buffer[256];
    sprintf(buffer,
        "CPU: %s\nClock: %d MHz\nFeatures: FPU (sim)\nCores: 1\nFamily: m68k\
n",
        get_cpu_name(), get_cpu_clock());
    data->data = strdup(buffer);
    data->size = strlen(buffer);
    data->position = 0;
    file->private_data = data;
    file->position = 0;
    file->read = proc_read;
    file->write = NULL;
    file->close = proc_close;
    file->ioctl = NULL;
    file->lseek = proc_lseek;
    return 0;


int proc_uptime_open(File *file, const char *path, int flags)
    ProcData *data = (ProcData*)malloc(sizeof(ProcData));
    if (!data) return -1;
    static int uptime = 0;
    uptime += 10;
    char buffer[64];
    sprintf(buffer, "%d seconds\n", uptime);
    data->data = strdup(buffer);
    data->size = strlen(buffer);
    data->position = 0;
    file->private_data = data;
    file->position = 0;
    file->read = proc_read;
    file->write = NULL;
    file->close = proc_close;
    file->ioctl = NULL;
    file->lseek = proc_lseek;
    return 0;
```

7.7 vfs_drivers.c

```
#include "vfs.h"
```

```
extern int tty_open(File *, const char *, int);
extern int tty_read(File *, void *, size_t);
extern int tty_write(File *, const void *, size_t);
extern int tty_close(File *);
extern int tty_ioctl(File *, int, void *);
extern size_t tty_lseek(File *, size_t, int);

extern int null_open(File *, const char *, int);
extern int null_read(File *, void *, size_t);
extern int null_write(File *, const void *, size_t);
extern int null_close(File *);
extern size_t null_lseek(File *, size_t, int);

extern int zero_open(File *, const char *, int);
extern int zero_read(File *, void *, size_t);
extern int zero_write(File *, const void *, size_t);
extern int zero_close(File *);
extern size_t zero_lseek(File *, size_t, int);

extern int proc_meminfo_open(File *, const char *, int);
extern int proc_cpuinfo_open(File *, const char *, int);
extern int proc_uptime_open(File *, const char *, int);
extern int proc_read(File *, void *, size_t);
extern int proc_close(File *);
extern size_t proc_lseek(File *, size_t, int);

DeviceDriver drivers[] =
    "/dev/tty", tty_open, tty_read, tty_write, tty_close, tty_ioctl,
tty_lseek,
    "/dev/null", null_open, null_read, null_write, null_close, NULL,
null_lseek,
    "/dev/zero", zero_open, zero_read, zero_write, zero_close, NULL,
zero_lseek,
    "/proc/meminfo", proc_meminfo_open, proc_read, NULL, proc_close, NULL,
proc_lseek,
    "/proc/cpuinfo", proc_cpuinfo_open, proc_read, NULL, proc_close, NULL,
proc_lseek,
    "/proc/uptime", proc_uptime_open, proc_read, NULL, proc_close, NULL,
proc_lseek,
    NULL, NULL, NULL, NULL, NULL, NULL, NULL
;
```

7.8 vfs.c

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "vfs.h"

static File *fd_table[256];
static int fd_count = 0;

extern DeviceDriver drivers[];

int allocate_fd(File *file)
```

```
    for (int i = 3; i < 256; i++)
        if (fd_table[i] == NULL)
            fd_table[i] = file;
            fd_count++;
            return i;


    return -1;


File *get_file_from_fd(int fd)
    if (fd < 0 || fd >= 256) return NULL;
    return fd_table[fd];


void free_fd(int fd)
    if (fd >= 0 && fd < 256)
        fd_table[fd] = NULL;
        fd_count--;



int vfs_open(const char *path, int flags)
    File *file = (File*)malloc(sizeof(File));
    if (!file) return -1;
    memset(file, 0, sizeof(File));
    file->name = strdup(path);
    file->type = FILE_TYPE_REGULAR;
    file->position = 0;
    file->ref_count = 1;
    extern int fat_open(File *file, const char *path, int flags);
    if (fat_open(file, path, flags) == 0)
        int fd = allocate_fd(file);
        if (fd >= 0) return fd;
        free(file->name); free(file); return -1;

    for (int i = 0; drivers[i].path != NULL; i++)
        if (strcmp(path, drivers[i].path) == 0)
            file->type = FILE_TYPE_DEVICE;
            file->read = drivers[i].read;
            file->write = drivers[i].write;
            file->close = drivers[i].close;
            file->ioctl = drivers[i].ioctl;
            file->lseek = drivers[i].lseek;
            if (drivers[i].open(file, path, flags) == 0)
                int fd = allocate_fd(file);
                if (fd >= 0) return fd;

            break;


    free(file->name); free(file);
    return -1;


int vfs_read(int fd, void *buffer, size_t size)
    File *file = get_file_from_fd(fd);
```

```
    if (!file) return -1;
    if (file->read) return file->read(file, buffer, size);
    return -1;


int vfs_write(int fd, const void *buffer, size_t size)
    File *file = get_file_from_fd(fd);
    if (!file) return -1;
    if (file->write) return file->write(file, buffer, size);
    return -1;


int vfs_close(int fd)
    File *file = get_file_from_fd(fd);
    if (!file) return -1;
    int result = 0;
    if (file->close) result = file->close(file);
    free(file->name); free(file);
    free_fd(fd);
    return result;


int vfs_ioctl(int fd, int cmd, void *arg)
    File *file = get_file_from_fd(fd);
    if (!file) return -1;
    if (file->ioctl) return file->ioctl(file, cmd, arg);
    return -1;


size_t vfs_lseek(int fd, size_t offset, int whence)
    File *file = get_file_from_fd(fd);
    if (!file) return (size_t)-1;
    if (file->lseek) return file->lseek(file, offset, whence);
    switch (whence)
        case 0: file->position = offset; break;
        case 1: file->position += offset; break;
        case 2: return (size_t)-1;
        default: return (size_t)-1;

    return file->position;


void vfs_init(void)
    memset(fd_table, 0, sizeof(fd_table));
    fd_count = 0;
    int fd0 = vfs_open("/dev/tty", O_RDWR);
    int fd1 = vfs_open("/dev/tty", O_RDWR);
    int fd2 = vfs_open("/dev/tty", O_RDWR);
    if (fd0 != 0)  fd_table[0] = fd_table[fd0]; fd_table[fd0] = NULL;
fd_count--;
    if (fd1 != 1)  fd_table[1] = fd_table[fd1]; fd_table[fd1] = NULL;
fd_count--;
    if (fd2 != 2)  fd_table[2] = fd_table[fd2]; fd_table[fd2] = NULL;
fd_count--;
```

7.9 shell.c

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "vfs.h"

typedef struct
    char *name;
    int (*func)(int argc, char **argv);
 Command;

int cmd_help(int argc, char **argv);
int cmd_echo(int argc, char **argv);
int cmd_cat(int argc, char **argv);
int cmd_ls(int argc, char **argv);
int cmd_cd(int argc, char **argv);
int cmd_mkdir(int argc, char **argv);
int cmd_reboot(int argc, char **argv);
int cmd_shutdown(int argc, char **argv);
int cmd_meminfo(int argc, char **argv);
int cmd_clear(int argc, char **argv);

Command commands[] =
    "help", cmd_help, "echo", cmd_echo, "cat", cmd_cat,
    "ls", cmd_ls, "cd", cmd_cd, "mkdir", cmd_mkdir,
    "reboot", cmd_reboot, "shutdown", cmd_shutdown,
    "meminfo", cmd_meminfo, "clear", cmd_clear,
    NULL, NULL
;

char *gets_line(char *buffer, int size)
    int i = 0; char c;
    while (i < size - 1)
        c = getchar();
        if (c == '' || c == '')  buffer[i] = ''; putchar(''); return buffer;
        else if (c == '' || c == 0x7F)  if (i > 0)  i--; putchar('');
putchar(' '); putchar('');
        else if (c >= 0x20 && c < 0x7F)  buffer[i++] = c; putchar(c);

    buffer[i] = ''; return buffer;


int execute_line(char *line)
    char *args[16]; int argc = 0; char *token; char line_copy[256];
    while (*line == ' ') line++;
    line[strcspn(line, "")] = 0; line[strcspn(line, "")] = 0;
    if (line[0] == '' || line[0] == '#') return 0;
    strcpy(line_copy, line);
    token = strtok(line_copy, " ");
    while (token != NULL && argc < 16)  args[argc++] = token; token =
strtok(NULL, " ");
    if (argc == 0) return 0;
    for (int i = 0; commands[i].name != NULL; i++)
        if (strcmp(commands[i].name, args[0]) == 0) return
commands[i].func(argc, args);
```

```
    vfs_write(1, "Comando desconhecido: ", 23);
    vfs_write(1, args[0], strlen(args[0])); vfs_write(1, "", 1);
    return -1;


int cmd_help(int argc, char **argv)
    vfs_write(1, "Comandos disponiveis:\n", 23);
    for (int i = 0; commands[i].name != NULL; i++)
        vfs_write(1, "  ", 2); vfs_write(1, commands[i].name,
strlen(commands[i].name)); vfs_write(1, "", 1);

    return 0;


int cmd_echo(int argc, char **argv)
    for (int i = 1; i < argc; i++)  vfs_write(1, argv[i], strlen(argv[i]));
if (i < argc - 1) vfs_write(1, " ", 1);
    vfs_write(1, "", 1); return 0;


int cmd_cat(int argc, char **argv)
    if (argc < 2)  vfs_write(1, "Uso: cat <arquivo>\n", 20); return -1;
    int fd = vfs_open(argv[1], O_RDONLY);
    if (fd < 0)  vfs_write(1, "Erro: nao foi possivel abrir ", 29);
vfs_write(1, argv[1], strlen(argv[1])); vfs_write(1, "", 1); return -1;
    char buffer[128]; int bytes;
    while ((bytes = vfs_read(fd, buffer, sizeof(buffer))) > 0)  vfs_write(1,
buffer, bytes);
    vfs_close(fd); return 0;


int cmd_ls(int argc, char **argv)
    DIR dir; FILINFO fno; const char *path = (argc > 1) ? argv[1] : "/";
    if (fopendir(&dir, path) != FR_OK)  vfs_write(1, "Erro: nao foi possivel
listar ", 30); vfs_write(1, path, strlen(path)); vfs_write(1, "", 1); return
-1;
    while (freaddir(&dir, &fno) == FR_OK && fno.fname[0] != 0)
        vfs_write(1, fno.fname, strlen(fno.fname));
        if (fno.fattrib & AM_DIR) vfs_write(1, "/", 1);
        vfs_write(1, "  ", 2);

    vfs_write(1, "", 1); fclosedir(&dir); return 0;


int cmd_cd(int argc, char **argv)
    if (argc < 2)  vfs_write(1, "cd: falta diretorio\n", 21); return -1;
    vfs_write(1, "cd: mudando para ", 18); vfs_write(1, argv[1],
strlen(argv[1])); vfs_write(1, "", 1); return 0;


int cmd_mkdir(int argc, char **argv)
    if (argc < 2)  vfs_write(1, "mkdir: falta diretorio\n", 24); return -1;
    FRESULT result = fmkdir(argv[1]);
    if (result != FR_OK)  vfs_write(1, "Erro ao criar diretorio\n", 25);
return -1;
    vfs_write(1, "mkdir: criado ", 15); vfs_write(1, argv[1],
```

```
strlen(argv[1])); vfs_write(1, "", 1); return 0;


int cmd_reboot(int argc, char **argv)  vfs_write(1, "Reiniciando sistema...\
n", 24); return 0;
int cmd_shutdown(int argc, char **argv)  vfs_write(1, "Desligando sistema...\
n", 23); return 0;

int cmd_meminfo(int argc, char **argv)
    int fd = vfs_open("/proc/meminfo", O_RDONLY);
    if (fd < 0)  vfs_write(1, "Erro: /proc/meminfo nao disponivel\n", 36);
return -1;
    char buffer[128]; int bytes;
    while ((bytes = vfs_read(fd, buffer, sizeof(buffer))) > 0)  vfs_write(1,
buffer, bytes);
    vfs_close(fd); return 0;


int cmd_clear(int argc, char **argv)
    int fd = vfs_open("/dev/tty", O_RDWR);
    if (fd >= 0)  vfs_ioctl(fd, TIOC_CLEAR, NULL); vfs_close(fd);
    return 0;


void run_boot_script(void)
    int fd = vfs_open("/boot.bat", O_RDONLY);
    if (fd < 0)  vfs_write(1, "Nenhum script de boot encontrado\n", 34);
return;
    vfs_write(1, "\n=== Executando /boot.bat ===\n", 31);
    char line[256]; int pos = 0; char c; int bytes;
    while ((bytes = vfs_read(fd, &c, 1)) > 0)
        if (c == '' || c == '')
            if (pos > 0)  line[pos] = ''; pos = 0;
                if (line[0] != '#')  vfs_write(1, "[boot] ", 7); vfs_write(1,
line, strlen(line)); vfs_write(1, "", 1); execute_line(line);
            continue;

        if (pos < sizeof(line) - 1) line[pos++] = c;

    if (pos > 0)  line[pos] = ''; if (line[0] != '#')  vfs_write(1, "[boot]
", 7); vfs_write(1, line, strlen(line)); vfs_write(1, "", 1);
execute_line(line);
    vfs_close(fd); vfs_write(1, "=== Script de boot finalizado ===\n\n", 36);


void shell_loop(void)
    char line[256];
    run_boot_script();
    vfs_write(1, "--- Shell do Orion68DOS ---\n", 24);
    vfs_write(1, "Digite 'help' para comandos\n\n", 30);
    while (1)  vfs_write(1, "> ", 2); gets_line(line, sizeof(line));
execute_line(line);


int main(void)  vfs_init(); shell_loop(); return 0;
```

8. COMPILAÇÃO E INSTALAÇÃO

8.1 Compilação

```
make clean
make
ls -la shell.elf
```

8.2 Estrutura de Diretórios

```
/
|-- boot.bat
|-- bin/
|   `-- shell.elf
|-- dev/
|   |-- tty
|   |-- null
|   `-- zero
|-- proc/
|   |-- meminfo
|   |-- cpuinfo
|   `-- uptime
|-- tmp/
`-- var/
```

9. TESTES E VALIDAÇÃO

9.1 Teste de Arquivos

```
echo "Hello World" > /tmp/test.txt
cat /tmp/test.txt
```

9.2 Teste de /proc

```
cat /proc/meminfo
cat /proc/cpuinfo
cat /proc/uptime
```

9.3 Teste de Boot Script

Arquivo /boot.bat:

```
echo "Orion68DOS iniciando..."
meminfo
echo "Sistema pronto!"
```

10. APªNDICES

10.1 Mapeamento de Flags

VFS Flag	Valor	FATFS Flag	Descrição
O_RDONLY	0x01	FA_READ	Somente leitura
O_WRONLY	0x02	FA_WRITE	Somente escrita
O_RDWR	0x03	FA_READ | FA_WRITE	Leitura e escrita
O_CREAT	0x04	FA_CREATE_ALWAYS	Criar se não existir
O_TRUNC	0x08	FA_CREATE_ALWAYS	Truncar se existir
O_APPEND	0x10	FA_OPEN_APPEND	Escrever no final

10.2 Descritores Padrão

FD	Nome	Dispositivo	Descrição
0	stdin	/dev/tty	Entrada padrão (teclado)
1	stdout	/dev/tty	Saída padrão (vídeo)
2	stderr	/dev/tty	Saída de erro (vídeo)

10.3 Comandos IOCTL

Comando	Valor	Descrição
TIOC_CLEAR	0x01	Limpa a tela
TIOC_GETXY	0x02	Obtém posição do cursor
TIOC_SETXY	0x03	Define posição do cursor

11. EXEMPLO DE SESSÃO COMPLETA

```
=== Executando /boot.bat ===
[boot] echo "Orion68DOS v1.0 iniciando..."
Orion68DOS v1.0 iniciando...
[boot] meminfo
MemTotal: 1024 KB
MemFree:  512 KB
=== Script de boot finalizado ===

--- Shell do Orion68DOS ---
> help
Comandos disponiveis:
  help echo cat ls cd mkdir reboot shutdown meminfo clear
> echo "Teste VFS"
Teste VFS
> cat /proc/meminfo
MemTotal: 1024 KB
MemFree: 512 KB
> cat /dev/null
> clear
(tela limpa)
> shutdown
Desligando sistema...
```

FIM DO DOCUMENTO

Orion68DOS - Um sistema operacional para m68k
Versão 1.0 - Agosto 2026